

(continued)

* Cursor

→ Cursor is based on Select Statement.

→ Cursor is created inside a procedure always.

→ We can use a ORDER BY clause with cursor.

e.g.

declare c1 cursor for select ename, sal from emp where deptno=1

ORDER BY sal desc; $\leftarrow$ CURSOR Definition / Declaration

(At this point the cursor doesn't contain any data)

* open c1 ; $\leftarrow$ This will open the cursor c1 and execute the SELECT Statement & it'll populate my cursor c1.

* Here, all the rows are brought in RAM in a single I/O and we can do processing there. and if we don't have enough RAM, it'll use Virtual Memory (HD-memory)

* We have a cursor pointer, which points at the 1st row in CURSOR;

* Close c1 ; $\leftarrow$ This will close the cursor c1 and free the server RAM;

* A CURSOR is a Read-Only variable, the data i.e. present in the CURSOR it can't be manipulated.

* The rows are fetched 1 row at a time into some intermediate a, b, c, d variables and do the processing with those variables.

* You can fetch Sequentially only (Top to Bottom)

* You can't fetch backwards in MySQL CURSOR.

* You can ONLY fetch 1 row at a time

* CURSOR has to be declared after all the variables.

//_

 c1

* fetch[↑] into a, b, c, d ; ← it fetches the next row

* If we make the while loop work 3 times:-

While x < 3 do

 fetch c1 into a, b, c, d ;

 /* Processing */

 insert into temp values (a, b);

 Set x = x + 1 ;

End while ;

* So, in SELECT statement if we use ORDER BY clause, then we would have done the processing for the top 3 rows ;

* Such report is known as RANKING REPORT

* But if we run loop more than the rows present in CURSOR, then it'll fetch the last row repeatedly

* To get the count of rows of emp table we use,

declare x int default 0 ;

declare y int ;

declare c1 cursor for select * from emp ;

select count(*) into y from emp ; ← returns # of rows in emp

open c1 ;

while x < y do

 --
 --
 --
end while ;

* Cursor Attributes

* Declare a CONTINUE handler for a NOT FOUND event.

* NOT FOUND is a cursor attribute, which returns a boolean TRUE value if the last fetch was ^{un}successful, and a boolean FALSE value if the last fetch was successful.

code:-

delimiter //

(create procedure abc)

begin

declare a int;

declare b varchar(15);

declare c int;

declare d int;

declare y int default 0;

declare c1 cursor for select * from emp;

~~open c1;~~ declare continue handler for not found set y = 1;

~~cursor-c1-loop: loop~~

open c1;

cursor-c1-loop: loop

fetch c1 into a, b, c, d;

if y = 1 then

leave cursor-c1-loop;

end if;

/* Processing */

insert into temp values (a, b);

end ~~cursor-c1-loop~~ loop;

close c1; ↳ not name of loop (just keyword loop)

end; //

delimiter *

Order of cursor Precedence :-

- i) declare variables
- ii) declare all cursors
- iii) declare continue handlers
- iv) open cursor & Logical processing
- v) close cursor
- vi) END of Program

* Open c1;

Open c1; ← ERROR

→ You cannot open the same cursor repeatedly, you'll get an error of cursor already open;

→ You'll have to close it before, to open it again;

* To reset the cursor pointer:-

→ close c1;

open c1;

* There is no upper limit on the number of CURSOR's in a procedure.

* Internally joins are working with the help of system CURSORS.
e.g.

cursor c1 is select * from emp;

cursor c2 is select * from dept;

declare continue for not found set y = 1;

open c1;

c1-loop: loop

Fetch^{c1} into a, b, c, d;

Open c2;

c2-loop: loop

Fetch c2;

```
close c2;
```

```
close c2;
```

```
END;
```

* Any program based on cursor, there are 5 steps which cannot be avoided :-

- 1) declaration of variable having same datatype & structure as table.
- 2) declaration of the cursor handler and a y variable.
- 3) Manually open the cursor
- 4) Manually fetch the rows into variables.
- 5) Close the cursor ;

Template code →

* Cursor based on computed column ($Sal * 12$), expression ($3sal + comm$), function, group by clause, having clause, order by clause, join, sub-query, set-operators, limit, offset, etc. is allowed.

* Practical Use

→ Used for storing / processing multiple rows.

*** → Used for LOCKING ROWS MANUALLY.

* To lock the rows manually :-

```
create Procedure abc()  
begin
```

```
declare c1 cursor for select * from emp for update;
```

```
open c1;
```

```
close c1;
```

```
end; //
```

```
delimiter ;
```

* (Now to lock rows, just call the procedure)

```
call abc ();
```

* Locks are automatically released when you rollback or commit;

* Parameters

* You can pass parameters to procedure (to make it flexible)

Types of Parameters :-

i) IN

ii) OUT

iii) INOUT

i) IN

→ Read Only

→

delimiter //

create procedure abc(~~in~~ y int)

begin

declare x int default 10;

~~exit abc~~ set y = 100; ~~---~~ ERROR

insert into temp values (y, 'inside abc');

end; //

delimiter ;

→ Can pass a constant, a variable or an expression

* mysql> call abc(5); ~~---~~ constant

mysql> set @x = 10;

mysql> call abc(@x); ~~---~~ variable

mysql> call abc(2 * @x + 10) ~~---~~ Expression

procedure

→ a ~~call~~ can call another procedure

delimiter //

create procedure pqr ()

begin

~~call~~ declare x int default 10;

call abc(5);

call abc(x);

call abc(2*x+5);

end; //

delimiter ;

mysql> call pqr();

* By default,

if a parameter is passed, it's an IN parameter by default.

* IN is the fastest in terms of processing speed.

ii) OUT

→ Write Only.

→ MOST SECURE

→ delimiter //

create procedure abc (out y int)

begin

Set y = 100;

end;

delimiter ;

→ delimiter //

```
create procedure pqr ()
```

```
begin
```

```
declare x int default 10;
```

```
insert into temp values (x, 'before abc');
```

```
set call abc(x);
```

```
insert into temp values (x, 'after abc');
```

```
end; //
```

```
delimiter ;
```

```
mysql> call pqr();
```

* Can pass variables only.

* Call by reference.

iii) INOUT

→ Read and Write only

→ Can pass variables only

→ Call by reference

→ BEST FUNCTIONALITY

→ delimiter //

```
create procedure abc (inout y int)
```

```
begin
```

```
Set y = y * y * y;
```

```
end; //
```

```
delimiter ;
```

→ delimiter //

```
create procedure pqr ()
```

```
begin
```

① declare ~~to~~ int default 10;

② insert into temp values (x, 'before abc');

③ call abc(x);

④ insert into temp values (x, 'after abc');

```
end;
```

```
delimiter ;
```

→ mysql> call pqr();

①

10

 x (Server RAM)
102489

② → insert (10, '---');

③ abc(10) → Set $x = 10 \times 10 \times 10$
passed by reference $x = 1000$

So, now

1000

 x (Server RAM)
102489

④ → insert (1000, '---')

* Procedure can return a value indirectly if you call by reference

* If you want to return a value on LOCAL network, e.g. HOME, WORK, etc. use INOUT parameter.

If you want to return value on PUBLIC network, e.g. Internet, WAN, etc. use OUT parameter (especially for sensitive data, e.g. OTP, Username, Password, etc)

* Stored Functions (stored objects)

* Objects that are stored in the DB
e.g. CREATE, ..., Tables, Indexes, Views, Procedures

* Anything that you do with create command is a stored object

(Stored Functions) ↓

* Routine that returns a value directly and compulsarily

* Stored in the database

* global functions

* Can be called through MySQL command line client, MySQL workbench, Java, .NET, or any Front-end BIW.

* Stored in the database in the COMPILED FORMAT;

You cannot call a function by itself.

mysql> call abc(); ←-- For calling SP;

mysql> Set x = pqr() ←-- For calling SF;

* Unlike a SP, a SF cannot be called by itself, because a SF returns a value and that value has to be stored somewhere;

And, therefore a SF has to be ~~either~~ equated with a variable or it has to be a part of some expression;

* SF can have IN parameters only.

SF are of 2 types

i) Deterministic

ii) Non-Deterministic

* → For the same i/p parameters, if the SF returns same result, it is considered **DETERMINISTIC**.

Otherwise, the SF is considered as **NON-DETERMINISTIC**.

* If you declare it incorrectly, the Stored Function may produce an unexpected result, or the available optimization is not used which degrades the performance.

* delimiter //

create function ABC()

returns int

deterministic

begin

return 10;

end; //

delimiter ;

}
Stored
Function

* SP

delimiter //

create procedure pqr ()

begin

declare x int;

Set x = abc ();

insert into temp values (x, 'after abc');

end; //

delimiter ;

Call Procedure:-

mysql> call pqr ();

* Functions with Parameter

delimiter //

create function abc (y int)

returns int

deterministic

begin

return $y * y$;

end; //

delimiter ;

Calling procedure:-

delimiter //

create procedure pqr ()

begin

declare x int;

Set x = abc (10);

insert into temp values (x, 'after abc');

end; //

delimiter ;

* Call procedure

```
mysql> call pqr();
```

* STORED FUNCTION CAN BE CALLED IN SELECT STATEMENTS
AND ALSO IN SQL COMMANDS

e.g. `Select abc(sal) from emp;`

`Delete from emp where abc(sal) = 10000;`

`Select abc(10) from emp;`

* To drop stored function:-

`> drop function function-name;`

EMP		TEMP	
ENAME	SAL	FIR	SEC
KING	9000		

* Stored function:-

`delimiter //`

`create function abc(y int)`

`returns boolean`

`deterministic`

`begin`

`if y > 5000 then`

`return TRUE;`

`else`

`return FALSE;`

`end if;`

`end; //`

`delimiter ;`

* Calling Procedure :-
(definition)

delimiter //

create procedure pqr ()

begin declare x int;
 Select sal into x from emp where ename = 'KING';
 if abc(x) then

 insert into temp values (x, '> 5000');

 else

 insert into temp values (x, '< 5000');

 end if;

end; //

delimiter ;

* Call procedure.

mysql> call pqr ();

* If the function is returning a boolean value then we can just use the function definition without equating to a variable in a "if condition"

* To see which all functions are available
> show function status; <-- all databases.

> show function status where db = 'Pune';

> show function status where name like 'a%';

* To see the source code of functions

> show create function abc;

To share the function with other users:-

```
dac1-mysql> create function abc() - - - - -;
```

```
root-mysql> grant execute on function puni.abc to dac2@localhost;
```

```
dac2-mysql> select puni.abc() from dual;
```

* Stored Object - Database Triggers

EMP			DEPTOT	
ENAME	SAL	DEPTNO	DEPTNO	SALTOT
A	5000	1	1	15000
B	5000	1	2	6000
C	5000	1		
D	3000	2		
E	3000	2		
			TEMPP	
			FIR	SEC

→ Triggers are available in some of the RDBMS

→ Trigger is a routine or a set of commands, that gets executed AUTOMATICALLY, whenever some events take place.

→ Triggers are written on TABLES

Events are :-

i) Before Insert, After Insert

ii) Before Update, After Update

iii) Before Delete, After Delete

} Total 6 events.

* Insert Trigger on ABC

delimiter // → trigger name (upto 30 chars)
create trigger abc
before insert → event
on emp for each row → on what table (ie. here emp table)
begin
inserted into temp values (1, 'inserted');
end; //
delimiter ;

* Uses

- maintain the logs of insertions
(such a log is known as audit trails)

* IF DML operations on table fails, then it'll cause the event to fail and then the trigger changes are AUTOMATICALLY ROLLBACKED.

* If the trigger fails, then it'll cause the event to fail, and then the DML operⁿ on table is AUTOMATICALLY ROLLBACKED.

* YOUR DATA WILL ALWAYS BE CONSISTENT.

→ To Maintain the logs,

AFTER INSERT TRIGGER is recommended ;

* Within the trigger, all MySQL-PL statements are allowed, e.g.
IF-Statement, Loops, Cursors, Sub-blocks, etc.

* You can call SP and SF inside the trigger.

* Rollback and Commit are not allowed inside the trigger, it has to be specified AFTERWARDS, at the end of transaction.

* If you Rollback or Commit Afterwards, your data will always be consistent;

* In MySQL, all the triggers are at row level, which means they will fire ONCE, for every row inserted.

* In MySQL, you can have 6 triggers per table.

* The triggers are at Server Level. (All of them) & you can perform the DML operations using any front-end S/W, the trigger will always execute.

* new.sal, new.ename, new.deptno are MySQL created variables, they can only be used within triggers.

begin

insert into temp values (new.sal, new.ename);

end; //

* Uses :-

i) automatic data duplication, automatic data mirroring, automatic data replication, concept of 11d Servers Standby database;

(AFTER INSERT TRIGGER is Recommended)

(SHADOW TABLES)

ii) Data Cleansing (BEFORE INSERT)

e.g. begin

Set new.ename = Upper(new.ename);

insert into temp values (new.sal, new.ename);

end; //

Data cleansing

iii) Validations (BEFORE INSERT)

* iv) Auto Updation of Related ~~Queries~~ Tables

e.g.

begin

update deptot set saltot = saltot + new.sal

where deptno = new.deptno;

end; //

* drop trigger abc; --> To Drop trigger

* If you drop the table, triggers and indexes are dropped automatically.

* If you drop the table, views, stored procedures, stored functions remain

* Show triggers; <-- Show all triggers in all schemas

Show triggers from db-name; <-- for specific db.

e.g. Show triggers from pune;

* `Select * from information-schema.triggers
where table_schema = 'pure' ;`

* Delete Trigger

→ `delimiter //`

`create trigger p90`

`before delete`

---> Event (before delete)

`on emp for each row`

`begin`

`insert into temp values (1, 'deleted');`

`end; //`

`delimiter ;`

Uses :-

i) maintain logs (audit trails) of deletions;

(AFTER DELETE is recommended)

* To access the ~~the~~ deleted row data, we can use (old.col-name)

`begin`

`insert into temp values (old.sal, old.ename)`

`end; //`

* old.sal, old.ename, old.deptno, old.empno are MySQL created Variables ;

ii) maintain HISTORY TABLE in the event of delete.
(AFTER DELETE is recommended)

iii) Auto updation of related tables;

e.g.

```
begin                                     (3000)
    update deptot set saltot = saltot - old.sal
    where deptno = old.deptno;
end; //                                  (2)
```

* Update Trigger

→ update emp set sal = 6000 where deptno = 1;

delimiter //

create trigger xyz

before update

--> Event (before update)

on emp for each row

begin

insert into temp values ('1', 'updated');

end; //

delimiter ;

* Uses :-

i) maintains logs of updations

(Such logs is called as audit trail)

(AFTER UPDATE is Recommended)

- * Here we have both old & new values from system variables.

begin

insert into temp values (old.sal, old.ename);

insert into temp values (new.sal, new.ename);

end; //

- > old.sal, old.ename, new.sal, new.ename, are MySQL Created variables.

- * maintain SHADOW and HISTORY tables in the event of update.
(AFTER UPDATE trigger is recommended)

* CASCADING TRIGGERS

- * One trigger causes a second trigger to execute which in turn causes a third trigger to execute, and so on.....
- * In MySQL, there is no upper limit on the number of levels for CASCADING TRIGGERS.
- * Mutating tables:- it's an ERROR, if some cascading trigger causes one of the previous triggers to execute, you'll get an error of MUTATING TABLES and the entire transaction is automatically rolled back.

* NORMALISATION (V-IMP)

- * applicable for RDBMS and ORDBMS
- * 1st to 4th normal form applicable for → RDBMS and ORDBMS
- * 5th to 9th normal form applicable for → ORDBMS
- * Normalisation is the concept of Table Design.
- * Which tables to create, what structure, which columns, datatypes, widths, and constraints.
- * Based on User Requirements.
- * Normalisation is a part of Design Phase.
- * Aim is to have an efficient Table Structure.

(Primary)
Aim

* Aim of Normalisation is to avoid Data Redundancy (avoid the unnecessary duplication of data)

- * Normalisation is done from an input perspective, i.e. its done from a forms perspective.

* VIEW THE ENTIRE APPLICATION ON PER TRANSACTION BASIS AND NORMALIZE EACH TRANSACTION SEPARATELY.

e.g. Customer places an order, Customer makes a payment, goods are delivered, Customer cancels the order.

Example :-

* CUSTOMER PLACES AN ORDER

→ Getting ready for Normalisation :-

Onum 5001 cname Subbram
 Cnum 1001 Ccity Pune Cpincode 411037
 Caddr MY
 Orderdate 13/9 Dellydate 16/9

Prodcd	Prodname	Qty	Rate	Itemtotal
P001	Marker	100	10	1000
P002	Battery	10	10	100
P003	Fan	100	20	2000

 OTotal

1) For a given transaction, make a list of fields

i) Onum	vi) Cpincode	xiv) OTotal
ii) Cnum	vii) Orderdate	xv) Qty
iii) Cname	viii) Dellydate	xvi) Rate
iv) Caddr	ix) Prodcd	xvii) Itemtotal
v) Ccity	x) Prodname	

2) Ask the client for some sample data

3) With the permission of client and their involvement, for every Column, strive for atomicity.

e.g. addr column → can't have multiple Sub columns like (plot no, bldg-name, floor, area, city pincode)

(columns are made up of sub-columns, sub-columns are made up of sub-sub columns)

- 4) For every ~~client~~ column, make a list of column properties and ask the User/Client to submit the same.

e.g.

Cpincode :- Numeric, No Decimal, Has to be +ve, 6-Digits, Mandatory/compulsory, Fixed length, etc.

Caddr :- Alphanumeric, variable-length, max. 50 character, etc.

Delydate :- Date, time, '00-MM-YYYY', '10:30 AM', etc.

- 5) Get the client Sign-Off.

- 6) End of client involvement.

- 7) For every column, assign the datatype (e.g. cpincode $\rightarrow$ unsigned int.)

- 8) For every column assign the maximum length.

e.g. Caddr $\rightarrow$ varchar (50);

- 9) Assign the NOT NULL, UNIQUE and CHECK constraints.

e.g. cpincode $\rightarrow$ NOT NULL

- 10) Remove the computed columns.

e.g. Itemtotal, Ottotal

(New List)

PK

- | | | |
|----------------|----------------|----------|
| i) <u>Onum</u> | vi) Cpincode | x) Qty |
| ii) Cnum | vii) Orderdate | xi) Rate |
| iii) Cname | viii) Deltdate | |
| iv) Caddr | ix) Prodd | |
| v) Ccity | x) Prodname | |

11) For all practical purposes you can have a single table with all these columns.

12) Key element will be the Primary Key of this table

* At this point of time, the data is in **Un-Normalised Form (UNF)**.

(Secondary Aim)

* Aim of Normalisation is to reduce the problem of INSERT, UPDATE, DELETE, etc.

* **Un-Normalised Form** → Starting point of Normalisation

Single Row Block

Cnum	☐			
Cnum	☐	Cname	☐	
Caddr	☐	ccity	☐	Cpincode ☐
Orderdate	☐	Deltdate	☐	

Multiple Row Block

Prodd	Prodname	Qty	Rate
---	---	---	---
---	---	---	---
---	---	---	---

* Columns in multiple row block, are called the Repeating Group.

* Step-1

Remove the repeating group in a new table.

<u>Onum</u>	<u>Prodcod</u>
Cnum	Prodname
Cname	Qty
Caddr	Rate
Ccity	
Cpincod	
Orderdate	
Delvdate	

* Step-2

Key element will be Primary Key of the table

PK →	<u>Onum</u>	<u>Prodcod</u> → PK
	Cnum	Prodname
	Cname	Qty
	Caddr	Rate
	Ccity	
	Cpincod	
	Orderdate	
	Delvdate	

* Step-3

This step may or may not be required

→ Add the primary key of the original table to the new table to give you a composite Primary Key.

Onum

Cnum

Cname

Caddr

Ccity

Cpincode

Orderdate

Delvdate

Onum

Prodcid

Prodname

Qty

Rate

} Composite PK

The above 3 steps are to be repeated again and again infinitely, till you cannot normalise any further

* **FIRST NORMAL FORM (FNF) (1NF) (Single Normal Form)**

→ Repeating groups are removed from table design.

→ 1: Many Relationship is always encountered in FIRST NORMAL FORM.

→ DEPT and EMP tables are in FIRST NORMAL FORM, because 1: Many relationship is encountered here.

(Till here our data is in 1NF)

PTO

* Step-4

Only the tables with Composite PK are examined

<u>T1</u>	<u>T2</u>	
---	<u>Onum</u>	} Key Elements
---	<u>Proded</u>	
---	Prodname	} Non-key Elements
---	Qty	
---	Rate	

Those columns / nonkey elements which

* Step-5

Those non-key elements that are not fully dependant (are partially dependant) on the entire Composite PK; they are to be removed into a new table.

<u>Onum</u>	<u>Onum</u>	<u>Proded</u>
Cnum	<u>Proded</u>	Prodname
Pname	Qty	Rate
Caddr		
Ccity		
Cpincode		
Orderdate		
Delvdate		

* Step-6

The key element of original table on which the non-key elements were fully dependent, it is to be added to the new table and it will be the PK of the new table.

Onum

Cnum

Cname

Ccaddr

Ccity

Cpincode

Orderdate

Delvdate

Onum

Prodcid

Qty

Prodcid

Prodname

Rate

→ Above 3 steps are to be repeated again and again infinitely till you cannot normalise further.

* Second Normal Form (2NF) (Double Normal form) (SNF)

→ Every column is functionally dependent on Primary Key
(This is known as FUNCTIONAL DEPENDENCY)

* FUNCTIONAL DEPENDENCY :- without the PK that column cannot function.

* Step - 7

Only the non key elements are examined.

* Step - 8

The interdependent non-key elements are to be removed in a separate table.

P.T.O

<u>Onum</u>	<u>Cnum</u>	<u>Onum</u>	<u>Proded</u>
Orderdate	Cname	<u>Proded</u>	Prodname
Delydate	Caddr	Qty	Rate
	Ccity		
	Cpincode		

* Step - 9

Key element of will be PK of new table, and the PK of new table, it is to be retained in the original table for relationship purposes

<u>Onum</u>	<u>Cnum</u>	<u>Onum</u>	<u>Proded</u>
Orderdate	Cname	<u>Proded</u>	Prodname
Delydate	Caddr	Qty	Rate
to maintains relationship --- Cnum	Ccity		
	Cpincode		

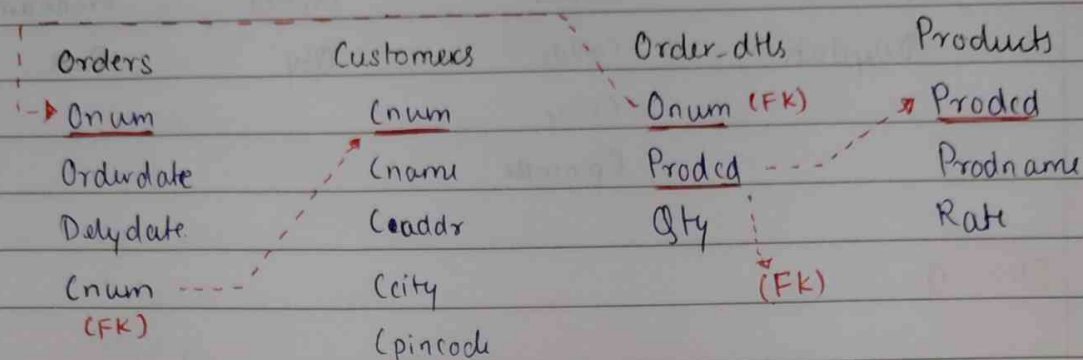
* Third Normal Form (3NF) (Triple Normal Form) (TNF)

→ Transitive dependencies are removed from table design (inter-dependencies)

ORDERS	CUSTOMERS	ORDER-DTLS	PRODUCTS
<u>Onum</u>	<u>Cnum</u>	<u>Onum</u>	<u>Proded</u>
Orderdate	Cname	<u>Proded</u>	Prodname
Delydate	Caddr	Qty	Rate
Cnum	Ccity		
	Cpincode		

child → Parent
(FK) (PK)

* PRIMARY KEY is a by-product of Normalisation.



* 4

* Fourth Normal Form (4NF)

- is an extension to 3rd Normal Form
- also known as Boyce-Codd Normal Form or BCNF
- You may or maynot implement the 4th normal form.
- used to protect the integrity of data
- Normally used on public Network (e.g. INTERNET)

CUSTOMERS

CNUM	CNAME	CADDR	CCITY	CPINCODE
1001	Sunbeam	MYD	Pune	411037
1002	YCPAIT	NPT	Mumbai	400021
1003	MET-IIT	BDR	Mumbai	400050
1004	Salman	YWD	Mumbai	411037

Problematic →

So, instead of storing Ccity & Cpincode in same table as customers, we'll create a new table "CPINCODE_CCCITY"

C-PINCODE-CCITY		Customers			(FK)
(PK) C-PINCODE	CCITY	Cnum	Cname	Caddr	CPincode
411037	Pune	---	---	---	---
400021	Mumbai	---	---	---	---
400050	Mumbai	---	---	---	---

* Post Normalisation

- 1) Add a few extra columns in each table (e.g. col1, col2, etc), to maintain the logs of insertion, updation and deletions. Typically store users, Sysdate, now, ip-address, etc.
- 2) Implement the extension columns in every table (min 30) (e.g. x1, x2, x3, etc.)

* De-Normalisation

- * If the data is large, if the select statements are slow, then we add an extra column to the table, to improve performance, to make the select statement more faster.
- * Normally done for computed columns, expressions, function-based columns, summary columns, formula columns, etc. (e.g. Itemtotal, Ottotal)

Orders	Customers	Order-dtls	Products
<u>Cnum</u>	<u>Cnum</u>	<u>Cnum</u>	<u>Prodcd</u>
Orderdate	Cname	Prodcd	Prodname
Qtydate	Caddr	Qty	Rate
Ototal	Ccity	Itemtotal	
Cnum	CPincode		

* In some case you may want to create an extra table and store the totals there

e.g. DEPTOT

DEPTNO	SALTOT
1	15000
2	6000

NoSQL (Not Only SQL)

* New Technology

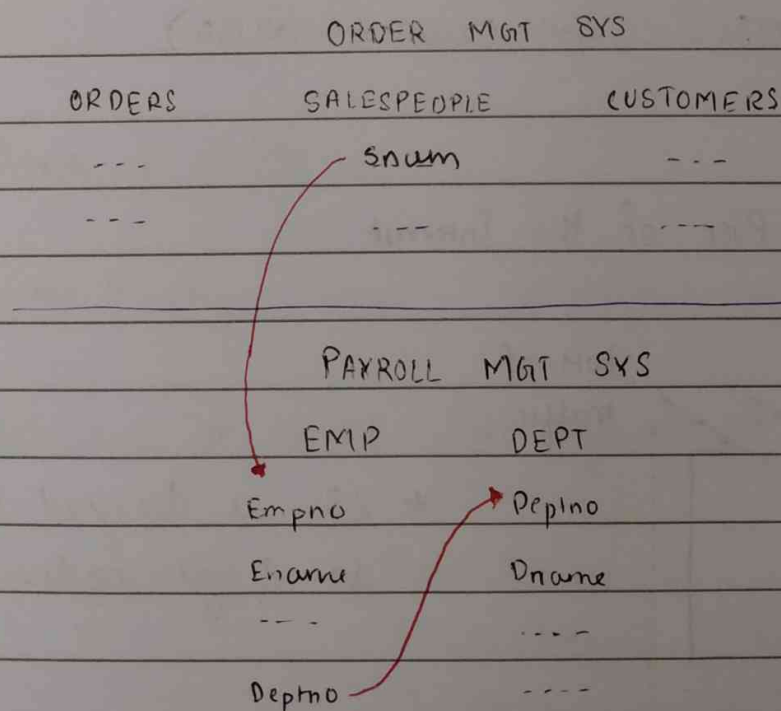
Type of database management system

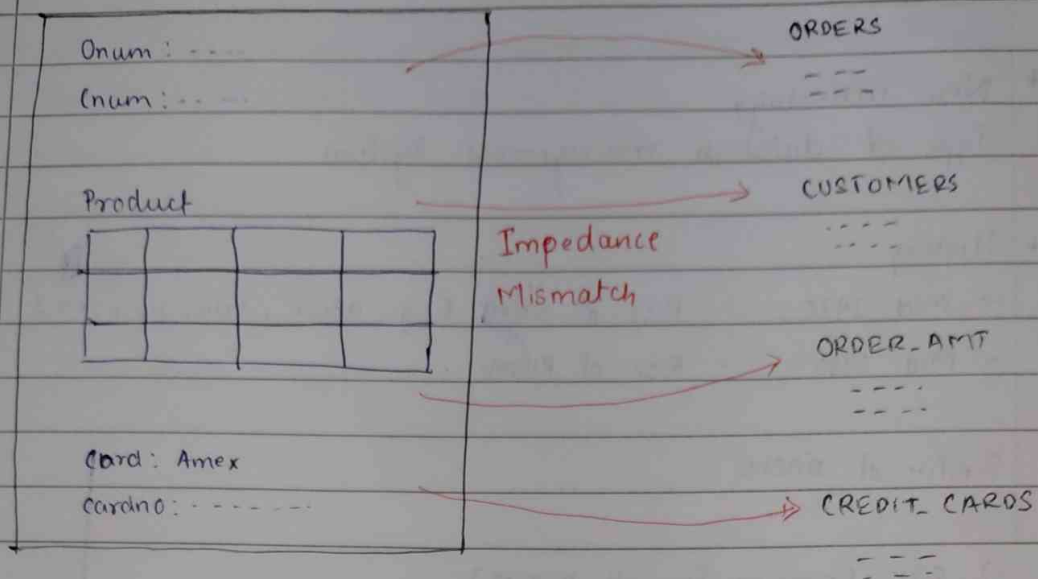
* History

- Mid 1970's - Rise of DBMS (e.g. dBase, Lotus-1-2-3, etc)
- Mid 1980's - Rise of RDBMS

* Benefits of RDBMS

- SQL (common for all RDBMS)
- Complex Transactions
- Excellent Reporting Tools available
- Data Persistence
- Integration mechanism between applications.





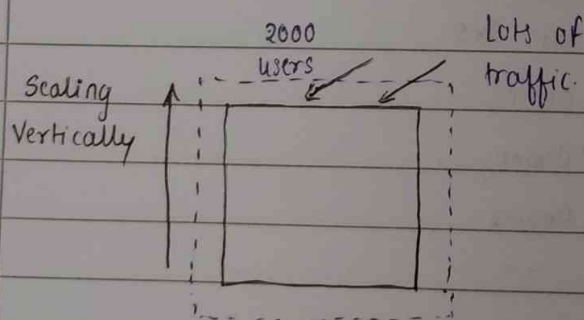
* Problem with RDBMS is Impedance Mismatch.

IMPEDANCE MISMATCH :- One logical group^{of} fields in the memory is splattered across multiple tables in the DB's.

Solution :- OODBMS (Object Oriented - DBMS)

* Problem :-

Early 2000's → Rise of the Internet



* SQL is designed to work on single node.

→ Scaling Vertically

a) Very expensive

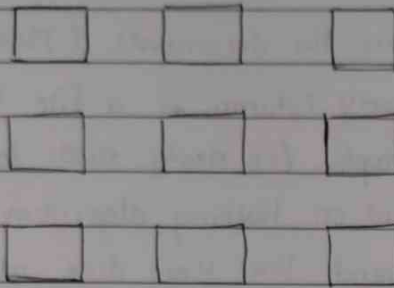
b) Very difficult to implement.

Google → Bigtable
Amazon → Dynamo

MongoDB
CouchDB
Project Voldemort
Hypertable
Cassandra
Dynamo

NOSQL

So, instead



Small
machines

→
Scaling Horizontally
(Grid of Servers)

* SQL does not work great with grid of servers or when horizontal scaling.

Solution:-
i) Scaling Horizontally
ii) Cloud Computing
iii) HAAS, SAAS, PAAS, IAAS

* Birth of NoSQL

Twitter Hashtag:- "nosql" → NoSQL Meetup

* Definition of NoSQL:- No definition of NoSQL.

* Characteristics:-
i) Open-Source
ii) Cluster-Friendly (ability to run on large clusters)
(scaling-horizontally)
iii) Non-Relational.
iv) Designed for 21st century Web (high-traffic web)
v) Schema-less

* Various Data Models:-

(e.g. mongo)

- i) Document Database → Store the documents (JSON-style or BSON)
- ii) Column-family Database → Every column is a file. (designed for fast Search)
- iii) Graph Database → Store graphs (e.g. Neo4j, NoSQL for the Enterprise)
- iv) Key-Value Database → Based on hashing algorithm
(value of the field it will generate the Hard disk address)
(e.g. project voldemort, A distributed DB, riak, redis)